

- C/C++

- 关键字

- C语言宏中"#"和"##"的用法
- 关键字volatile有什么含意？并举出三个不同的例子？
- 关键字static的作用是什么？
- 在C语言中，为什么 static变量只初始化一次？
- extern"C" 的作用是什么？
- const有什么作用？
- 什么情况下使用const关键字？
- new/delete与malloc/free的区别是什么？
- strlen("\0") = ? sizeof("\0")=?
- sizeof和strlen有什么区别？
- 不使用 sizeof，如何求int占用的字节数？
- C语言中 struct与 union的区别是什么？
- 左值和右值是什么？
- 什么是短路求值？
- ++a和a++有什么区别？两者是如何实现的？

- 内存

- C语言中内存分配的方式有几种？
- 堆与栈有什么区别？
- 栈在C语言中有什么作用？
- C语言函数参数压栈顺序是怎样的？
- C++如何处理返回值？
- C++中拷贝赋值函数的形参能否进行值传递？
- C++的内存管理是怎样的？
- 什么是内存泄漏？
- 如何判断内存泄漏？

- 指针

- 数组指针和指针数组有什么区别？
- 函数指针和指针函数有什么区别？
- 数组名和指针的区别与联系是什么？
- 指针进行强制类型转换后与地址进行加法运算，结果是什么？
- 常量指针，指向常量的指针，指向常量的常量指针有什么区别？
- 指针和引用的异同是什么？如何相互转换？
- 野指针是什么？
- 如何避免野指针？
- C++中的智能指针是什么？
- 智能指针的内存泄漏如何解决？

- 预处理

- 预处理器标识#error的目的是什么？
- 定义常量谁更好？# define还是 const？
- typedef和 define有什么区别？
- 如何使用 define声明个常数，用以表明1年中有多少秒（忽略闰年问题）
- # include< filename. h>和# nclude" filename. h"有什么区别？
- 头文件的作用有哪些？
- 在头文件中定义静态变量是否可行，为什么？

- 写一个"标准"宏MIN，这个宏输入两个参数并返回较小的一个？
- 不使用流程控制语句，如何打印出1~1000的整数？
- 变量
 - 全局变量和静态变量的区别是什么？
 - 全局变量可不可以定义在可被多个.C文件包含的头文件中？为什么？
 - 局部变量能否和全局变量重名？
- 函数
 - 请写个函数在main函数执行前先运行
 - 为什么析构函数必须是虚函数？
 - 为什么C++默认的析构函数不是虚函数？
 - C++中析构函数的作用？
 - 静态函数和虚函数的区别？
 - 重载和覆盖有什么区别？
 - 虚函数表具体是怎样实现运行时多态的？
 - C语言是怎么进行函数调用的？
 - 请你说一说select
 - 请你说说fork,wait,exec函数
- 数组
 - 以下代码表示什么意思？
 - 数组下标可以为负数吗？
- 位操作
 - 如何求解整型数的二进制表示中1的个数？
 - 如何求解二进制中0的个数
 - 交换两个变量的值，不使用第三个变量。即a=3,b=5,交换之后a=5,b=3;
 - 给定一个整型变量a，写两段代码，第一个设置a的bit 3，第二个清除a的bit 3。在以上两个操作中，要保持其它位不变。
- 容器和算法
 - map和set有什么区别？分别又是怎么实现的？
 - STL的allocator有什么作用？
 - STL迭代器如何删除元素？
 - STL中MAP数据如何存放的？
 - STL中map与unordered_map有什么区别？
 - vector和list的区别是什么？
 - STL中迭代器有什么作用？有指针为何还要迭代器？
 - epoll的原理是什么？
 - STL里resize和reserve的区别是什么？
- 类和数据抽象
 - C++中类成员的访问权限？
 - C++中struct和class的区别是什么？
 - C++类内可以定义引用数据成员吗？
 - 面向对象与泛型编程是什么？
 - 什么是右值引用，跟左值又有什么区别？
 - 析构函数可以为virtual型，构造函数则不能，为什么？
 - C++中空类默认产生哪些类成员函数？
- 面向对象
 - 面向对象和面向过程有什么区别？

- 面向对象的基本特征有哪些？
- 什么是深拷贝？什么是浅拷贝？
- 什么是友元？
- 基类的构造函数/析构函数是否能被派生类继承？
- 初始化列表和构造函数初始化的区别？
- C++中有那些情况只能用初始化列表，而不能用赋值？
- 类的成员变量的初始化顺序是什么？
- 当一个类为另一个类的成员变量时，如何对其进行初始化？
- C++能设计实现一个不能被继承的类吗？
- 构造函数没有返回值，那么如何得知对象是否构造成功？
- Public继承、protected继承、private继承的区别？
- C++提供默认参数的函数吗？
- 虚函数
 - 什么是虚函数？
 - C++如何实现多态？
 - 纯虚函数指的是什么？
 - 什么函数不能声明为虚函数？
 - C++中如何阻止一个类被实例化？

C/C++

关键字

C语言宏中"#"和"##"的用法

1. (#) 字符串化操作符

作用：将宏定义中的传入参数名转换成用一对双引号括起来参数名字符串。其只能用于有传入参数的宏定义中，且必须置于宏定义体中的参数名前。

如：

```
#define example( instr ) printf( "the input string is:\t%s\n", #instr )
#define example1( instr ) #instr当使用该宏定义时：
```

```
example( abc ); // 在编译时将会展开成：printf("the input string is:\t%s\n","abc")
string str = example1( abc ); // 将会展成：string str="abc"
```

2. (##) 符号连接操作符

作用：将宏定义的多个形参转换成一个实际参数名。

如：

```
#define exampleNum( n )  num##n
```

使用：

```
int num9 = 9;
int num = exampleNum( 9 ); // 将会扩展成 int num = num9
```

注意：

a. 当用##连接形参时，##前后的空格可有可无。

如：

```
#define exampleNum( n )      num ## n
// 相当于 #define exampleNum( n )      num##n
```

b. 连接后的实际参数名，必须为实际存在的参数名或是编译器已知的宏定义。

c. 如果##后的参数本身也是一个宏的话，##会阻止这个宏的展开。

```
#include <stdio.h>
#include <string.h>

#define STRCPY(a, b)  strcpy(a ## _p, #b)
int main()
{
    char var1_p[20];
    char var2_p[30];
    strcpy(var1_p, "aaaa");
    strcpy(var2_p, "bbbb");
    STRCPY(var1, var2);
    STRCPY(var2, var1);
    printf("var1 = %s\n", var1_p);
    printf("var2 = %s\n", var2_p);

    //STRCPY(STRCPY(var1,var2),var2);
    //这里是否会展开为： strcpy(strcpy(var1_p,"var2")_p,"var2")？答案是否定的：
    //展开结果将是：  strcpy(STRCPY(var1,var2)_p,"var2")
    //## 阻止了参数的宏展开！如果宏定义里没有用到 # 和 ##，宏将会完全展开
    // 把注释打开的话，会报错:implicit declaration of function 'STRCPY'
    return 0;
}
```

结果：

```
var1 = var2
var2 = var1
```

关键字volatile有什么含意？并举出三个不同的例子？

1. **并行设备的硬件寄存器**。存储器映射的硬件寄存器通常加volatile，因为寄存器随时可以被外设硬件修改。当声明指向设备寄存器的指针时一定要用volatile，它会告诉编译器不要对存储在这个地址的数据进行假设。
2. **一个中断服务程序中修改的供其他程序检测的变量**。volatile提醒编译器，它后面所定义的变量随时都有可能改变。因此编译后的程序每次需要存储或读取这个变量的时候，**都会直接从变量地址中读取数据**。如果没有volatile关键字，则编译器可能优化读取和存储，可能暂时使用寄存器中的值，如果这个变量由别的程序更新了的话，将出现不一致的现象。
3. **多线程应用中被几个任务共享的变量**。单地说就是防止编译器对代码进行优化。比如如下程序：

```
XBYTE[2]=0x55;
XBYTE[2]=0x56;
XBYTE[2]=0x57;
XBYTE[2]=0x58;
```

对外部硬件而言，上述四条语句分别表示不同的操作，会产生四种不同的动作，但是编译器却会对上述四条语句进行优化，**认为只有XBYTE[2]=0x58（即忽略前三条语句，只产生一条机器代码）**。如果键入volatile，编译器会逐一的进行编译并产生相应的机器代码（产生四条代码）。

关键字static的作用是什么？

1. 在函数体，**只会被初始化一次**，一个被声明为静态的变量在这一函数被调用过程中维持其值不变。
2. 在模块内（但在函数体外），一个被声明为**静态的变量**可以被模块内所用函数访问，但不能被模块外其它函数访问。它是一个**本地的全局变量**（只能被当前文件使用）。
3. 在模块内，一个被声明为**静态的函数**只可被这一模块内的其它函数调用。那就是，这个函数被限制在声明它的模块的本地范围内使用（**只能被当前文件使用**）。

在C语言中，为什么 static变量只初始化一次？

对于所有的对象（不仅仅是静态对象），**初始化都只有一次**，而由于静态变量具有“记忆”功能，初始化后，一直都没有被销毁，**都会保存在内存区域中**，所以不会再次初始化。存放在静态区的变量的生命周期一般比较长，它与整个程序**“同生死、共存亡”**，所以它只需初始化一次。而auto变量，即自动变量，由于它**存放在栈区**，一旦函数调用结束，就会**立刻被销毁**。

extern“C”的作用是什么？

extern "C"的主要作用就是为了能够正确实现C++代码调用其他C语言代码。加上extern "C"后，会**指示编译器这部分代码按C语言的进行编译**，而不是C++的。

const有什么作用？

1. 定义变量（局部变量或全局变量）为常量，例如：

```
const int N=100; //定义一个常量N
N=50; //错误，常量的值不能被修改
const int n; //错误，常量在定义的时候必须初始化
```

2. 修饰函数的参数，表示在函数体内不能修改这个参数的值。
3. 修饰函数的返回值。

a.如果给用 const修饰返回值的类型为指针，那么函数返回值（即指针）的内容是不能被修改的，而且这个返回值只能赋给被 const修饰的指针。例如：

```
const char GetString() //定义一个函数
char *str= GetString() //错误，因为str没有被 const修饰
const char *str=GetString() //正确
```

b.如果用 const修饰普通的返回值，如返回int变量，由于这个返回值是一个临时变量，在函数调用结束后这个临时变量的生命周期也就结束了，因此把这些返回值修饰为 const是没有意义的。

4. 节省空间，避免不必要的内存分配。例如：

```
#define PI 3.14159 //该宏用来定义常量
const double Pi=3.14159 //此时并未将P放入只读存储器中
double i=Pi //此时为Pi分配内存，以后不再分配
double I=PI //编译期间进行宏替换，分配内存
double j=Pi //没有内存分配再次进行宏替换，又一次分配内存
```

什么情况下使用const关键字？

1. 修饰一般常量。一般常量是指简单类型的常量。这种常量在定义时，修饰符const可以用在类型说明符前，也可以用在类型说明符后。例如：

```
int const x=2; const int x=2
```

2. 修饰常数组。定义或说明一个常数组可以采用如下格式：

```
int const a[8]={1,2,3,4,5,6,7,8}
const int a[8]={1,2,3,4,5,6,7,8}
```

3. 修饰常对象。常对象是指对象常量，定义格式如下：

```
class A:
const A a:
A const a:
```

定义常对象时，同样要进行初始化，并且该对象不能再被更新。修饰符 `const` 可以放在类名后面，也可以放在类名前面。

4. 修饰常指针

```
const int*p; //常量指针，指向常量的指针。即p指向的内存可以变，p指向的数值内容不可变
int const*p; //同上
int*const p; //指针常量，本质是一个常量，而用指针修饰它。即p指向的内存不可以变，但是p内存位置的数值可以变
const int* const p; //指向常量的常量指针。即p指向的内存和数值都不可变
```

5. 修饰常引用。被 `const` 修饰的引用变量为常引用，一旦被初始化，就不能再指向其他对象了。

6. 修饰函数的常参数。 `const` 修饰符也可以修饰函数的传递参数，格式如下：

```
void Fun (const int Var)
```

告诉编译器 `Var` 在函数体中不能被改变，从而防止了使用者一些无意的或错误的修改。

7. 修饰函数的返回值。 `const` 修饰符也可以修饰函数的返回值，表明该返回值不可被改变，格式如下：

```
const int FunI ();
const MyClass Fun2 ();
```

8. 在另一连接文件中引用 `const` 常量。使用方式有

```
extern const int 1:
extern const int j=10;
```

new/delete与malloc/free的区别是什么？

1. `new`、`delete` 是 C++ 中的操作符，而 `malloc` 和 `free` 是标准库函数。
2. 对于非内部数据对象来说，只使用 `malloc` 是无法完成动态对象要求的，一般在创建对象时需要调用构造函数，对象消亡时，自动的调用析构函数。而 `malloc` `free` 是库函数而不是运算符，不在编译器控制范围之内，**不能够自动调用构造函数和析构函数**。而 `NEW` 在为对象申请分配内存空间时，可以自动调用构造函数，同时也可以完成对对象的初始化。同理，`delete` 也可以自动调用析构函数。而 `malloc` **只是做一件事，只是为变量分配了内存，同理，`free` 也只是释放变量的内存**。

3. new返回的是指定类型的指针，并且可以自动计算所申请内存的大小。而 malloc需要我们计算申请内存的大小，并且在返回时强行转换为实际类型的指针。

strlen("\0") = ? sizeof("\0")=?

strlen("\0") =0, sizeof("\0")=2。

strlen用来计算字符串的长度（在C/C++中，字符串是以"\0"作为结束符的），它从内存的某个位置（可以是字符串开头，中间某个位置，甚至是某个不确定的内存区域）开始扫描直到碰到第一个字符串结束符\0为止，然后返回计数器值sizeof是C语言的关键字，它以字节的形式给出了其操作数的存储大小，操作数可以是一个表达式或括在括号内的类型名，操作数的存储大小由操作数的类型决定。

sizeof和strlen有什么区别？

strlen与 sizeof的差别表现在以下5个方面。

1. sizeof是运算符（是不是被弄糊涂了？事实上，sizeof既是关键字，也是运算符，但不是函数），而strlen是函数。sizeof后如果是类型，则必须加括弧，如果是变量名，则可以不加括弧。
2. sizeof运算符的结果类型是 size_t，它在头文件中 typedef为 unsigned int类型。该类型保证能够容纳实现所建立的最大对象的字节大小
3. sizeof可以用类型作为参数，strlen只能用char*作参数，而且必须是以"0结尾的。sizeof还可以以函数作为参数，如int g（），则 sizeof（g（））的值等于 sizeof（int）的值，在32位计算机下，该值为4。
4. 大部分编译程序的 sizeof都是在编译的时候计算的，所以可以通过 sizeof（x）来定义数组维数。而strlen则是在运行期计算的，用来计算字符串的实际长度，不是类型占内存的大小。例如，char str[20] = "0123456789"，字符数组str是编译期大小已经固定的数组，在32位机器下，为 sizeof（char）*20=20，而其 strlen大小则是在运行期确定的，所以其值为字符串的实际长度10。
5. 当数组作为参数传给函数时，传递的是指针，而不是数组，即传递的是数组的首地址。

不使用 sizeof，如何求int占用的字节数？

```
#include <stdio.h>
#define MySizeof(Value) (char *)(&value+1)-(char*)&value
int main()
{
    int i ;
    double f;
    double *q;
    printf("%d\r\n",MySizeof(i));
    printf("%d\r\n",MySizeof(f));
    printf("%d\r\n",MySizeof(a));
    printf("%d\r\n",MySizeof(q));
    return 0;
}
```

输出为：

4 8 32 4

上例中，`(char*) & Value`返回 `Value`的地址的第一个字节，`(char*) (& Value+1)` 返回`value`的地址的下一个地址的第一个字节，所以它们之差为它所占的字节数。

C语言中 struct与 union的区别是什么？

`struct` (结构体) 与 `union` (联合体) 是C语言中两种不同的数据结构，两者都是常见的复合结构，其区别主要表现在以下两个方面。

1. 结构体与联合体虽然都是由多个不同的数据类型成员组成的，但不同之处在于联合体中所有成员**共用一块地址空间**，即联合体只存放了一个被选中的成员，而结构体中所有成员占用空间是**累加**的，其所有成员都存在，不同成员会存放在不同的地址。在计算一个结构型变量的总长度时，其内存空间大小等于所有成员长度之和（需要考虑字节对齐），而在联合体中，所有成员不能同时占用内存空间，它们不能同时存在，所以**一个联合型变量的长度等于其最长的成员的长度**。
2. 对于联合体的不同成员赋值，**将会对它的其他成员重写**，原来成员的值就不存在了，而对结构体的不同成员赋值是互不影响的。

举个例子。下列代码执行结果是多少？

```
typedef union {double i; int k[5]; char c;}DATE;
typedef struct data( int cat; DATE cow;double dog;) too;
DATE max;
printf ("%d", sizeof(too)+sizeof(max));
```

假设为32位机器，`int`型占4个字节，`double`型占8个字节，`char`型占1个字节，而`DATE`是一个联合型变量，联合型变量共用空间，`uion`里面最大的变量类型是`int[5]`，**所以占用20个字节，它的大小是20**，而由于 `union`中`double`占了8个字节，因此 `union`是要8个字节对齐，**所占内存空间为8的倍数**。为了实现8个字节对齐，**所占空间为24**。而`data`是一个结构体变量，每个变量分开占用空间，依次为 `sizeof (int) + sizeof (DATE) + sizeof (double)` = 4+24+8=36按照8字节对齐，占用空间为40，所以结果为40+24=64。

左值和右值是什么？

左值是指可以出现在等号左边的变量或表达式，它最重要的特点就是**可写**（可寻址）。也就是说，它的值**可以被修改**，如果一个**变量或表达式的值不能被修改**，那么它就不能作为左值。

右值是指**只可以出现在等号右边的变量或表达式**。它最重要的特点是**可读**。一般的使用场景都是把一个右值赋值给一个左值。

通常，左值可以作为右值，但是右值不一定是左值。

什么是短路求值？

```
#include <stdio.h>
int main()
{
```

```

int i = 6;
int j = 1;
if(i>0||(j++)>0);
printf("%D\r\n",j);
return 0;
}

```

输出结果为1。

输出为什么不是2，而是1呢？其实，这里就涉及一个短路计算的问题。由于i语句是个条件判断语句，里面是有两个简单语句进行或运算组合的复合语句，因为或运算中，只要参与或运算的两个表达式的值都为真，则整个运算结果为真，而由于变量i的值为6，已经大于0了，而该语句已经为true，则不需要执行后续的j++操作来判断真假，所以后续的j++操作不需要执行，j的值仍然为1。

因为短路计算的问题，对于&&操作，由于在两个表达式的返回值中，如果有一个为假则整个表达式的值都为假，如果前一个语句的返回值为 false，则无论后一个语句的返回值是真是假，整个条件判断都为假，不用执行后一个语句，而a>b的返回值为 false，程序不执行表达式n=c>d，所以，n的值保持为初值2。

++a和a++有什么区别？两者是如何实现的？

a++的具体运算过程为

```

int temp = a;
a=a+1;
return temp;

```

++a的具体运算过程为

```

a=a+1;
return a;

```

后置自增运算符需要把原来变量的值复制到一个**临时的存储空间**，等运算结束后才会返回这个临时变量的值。所以前置自增运算符效率比后置自增要高

内存

C语言中内存分配的方式有几种？

1. 静态存储区分配

内存分配在程序编译之前完成，且在程序的整个运行期间都存在，例如全局变量、静态变量等。

2. 栈上分配

在函数执行时，函数内的局部变量的存储单元在栈上创建，函数执行结束时这些存储单元自动释放。

3. 堆上分配

堆与栈有什么区别？

1. 申请方式

栈的空间由操作系统自动分配/释放，堆上的空间手动分配/释放。

2. **申请大小的限制** 栈空间有限。在Windows下,栈是向**低地址**扩展的数据结构，是一块**连续的内存的区域**。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的，在WINDOWS下，栈的大小是2M（也有的说是1M，总之是一个编译时就确定的常数），如果申请的空间超过栈的剩余空间时，将提示overflow。因此，**能从栈获得的空间较小**

堆是很大的自由存储区。堆是向**高地址**扩展的数据结构，是不连续的内存区域。这是由于系统是用**链表来存储的空闲内存地址的**，自然是不连续的，而链表的遍历方向是由低地址向高地址。堆的大小**受限于计算机系统中有效的虚拟内存**。由此可见，堆获得的空间比较灵活，也比较大。

3. **申请效率** 栈由系统自动分配，速度较快。但程序员是无法控制的。

堆是由new分配的内存，一般速度比较慢，而且容易产生内存碎片,不过用起来最方便.

栈在C语言中有什么作用？

1. C语言中栈用来存储临时变量，临时变量包括**函数参数和函数内部定义的临时变量**。函数调用中和函数调用相关的**函数返回地址**，函数中的临时变量，寄存器等均保存在栈中，函数调用返回后从栈中恢复寄存器和临时变量等函数运行场景。
2. 多线程编程的基础是栈，**栈是多线程编程的基石**，每一个线程都最少有一个自己专属的栈，用来存储本线程运行时各个函数的临时变量和维系函数调用和函数返回时的函数调用关系和函数运行场景。操作系统最基本的功能是支持多线程编程，支持中断和异常处理，每个线程都有专属的栈，中断和异常处理也具有专属的栈，栈是操作系统多线程管理的基石。

C语言函数参数压栈顺序是怎样的？

从右至左。

C语言参数入栈顺序的好处就是可以动态变化参数个数。自左向右的入栈方式，最前面的参数被压在栈底。除非知道参数个数，否则是无法通过栈指针的相对位移求得最左边的参数。这样就变成了左边参数的个数不确定，正好和动态参数个数的方向相反。因此，C语言函数参数采用自右向左的入栈顺序，主要原因是为了支持可变长参数形式。

C++如何处理返回值？

C++函数返回可以按值返回和按常量引用返回，偶尔也可以按引址返回。多数情况下不要使用引址返回。

C++中拷贝赋值函数的形参能否进行值传递？

不能。如果是这种情况下，调用拷贝构造函数的时候，首先要将实参传递给形参，这个传递的时候又要调用拷贝构造函数(aa = ex.aa; //此处调用拷贝构造函数)。如此循环，无法完成拷贝，栈也会满。

```
class Example
{
public:
```

```

    Example(int a):aa(a) {}           //构造函数

    Example(Example &ex)               //拷贝构造函数 ( 引用传递参数 )
    {
        aa = ex.aa;                  //此处调用拷贝构造函数
    }
private:
    int aa;
};

int main()
{
    Example e1(10);
    Example e2 = e1;

    return 0;
}

```

C++的内存管理是怎样的？

在C++中，虚拟内存分为**代码段**、**数据段**、**BSS段**、**堆区**、**文件映射区**以及**栈区**六部分。

代码段：包括只读存储区和文本区，其中只读存储区存储字符串常量，文本区存储程序的机器代码。

数据段：存储程序中已初始化的全局变量和静态变量

BSS 段：存储未初始化的全局变量和静态变量（局部+全局），以及所有被初始化为0的全局变量和静态变量。

堆区：调用new/malloc函数时在堆区动态分配内存，同时需要调用delete/free来手动释放申请的内存。

映射区:存储动态链接库以及调用mmap函数进行的文件映射

栈：使用栈空间存储函数的返回地址、参数、局部变量、返回值

什么是内存泄漏？

简单地讲就是申请了一块内存空间，使用完毕后没有释放掉。

它的一般表现方式是程序运行时间越长，占用内存越多，最终用尽全部内存，整个系统崩溃。由程序申请的一块内存，且没有任何一个指针指向它，那么这块内存就泄露了。

如何判断内存泄漏？

1. 良好的编码习惯，尽量在涉及内存的程序段，检测出内存泄露。当程式稳定之后，再来检测内存泄露时，无疑增加了排除的困难和复杂度。使用了内存分配的函数，一旦使用完毕,要记得要使用其相应的函数释放掉。
2. 将分配的内存的指针以链表的形式自行管理，使用完毕之后从链表中删除，程序结束时可检查改链表。
3. Boost 中的smart pointer。
4. 一些常见的工具插件，如ccmalloc、Dmalloc、Leaky等等。

指针

数组指针和指针数组有什么区别？

数组指针就是指向数组的指针，它表示的是一个指针，这个指针指向的是一个数组，它的重点是指针。例如，`int (*pa) [8]`声明了一个指针，该指针指向了一个有8个int型元素的数组。下面给出一个数组指针的示例。

```
#include <stdio. h>
#include <stdlib. h>
void main()
{
    int b[12]={1,2,3,4,5,6,7,8,9,10,11,12};
    int (*p)[4];
    p = b;
    printf ("%d\n",  **(++p));
}
```

程序的输出结果为 5。

上例中，p是一个数组指针，它指向一个**包含有4个int类型数组**的指针，刚开始p被初始化为指向数组b的首地址，++p相当于把p所指向的地址**向后移动4个int所占用的空间**，此时p指向数组{5,6,7,8}，语句* (++p)；表示的是这个数组中**第一个元素**的地址(可以理解p为指向二维数组的指针，{ 1,2,3,4}， {5,6,7,8}， {9,10,11,12}。p指向的就是 { 1,2,3,4 } 的地址，*p就是指向元素，{ 1,2,3,4}，**p指向的就是1)，语句** (++p) 会输出这个数组的第一个元素5。

指针数组表示的是一个数组，而数组中的元素是指针。下面给出另外一个指针数组的示例

```
#include <stdio.h>
int main()
{
    int i;
    int *p[4];
    int a[4]={1,2,3,4};
    p[0] = &a[0];
    p[1] = &a[1];
    p[2] = &a[2];
    p[3] = &a[3];
    for(i=0;i<4;i++)
        printf ("%d",*p[i]);
    printf("\n");
    return 0;
}
```

程序的输出结果为1234。

函数指针和指针函数有什么区别？

1. 函数指针

如果在程序中定义了一个函数，那么在编译时系统就会为这个函数代码分配一段存储空间，这段存储空间的首地址称为这个**函数的地址**。而且函数名表示的就是这个地址。既然是地址我们就可以定义一个指针变量来存放，这个指针变量就叫作函数指针变量，简称**函数指针**。

```
int(*p)(int, int);
```

这个语句就定义了一个指向函数的指针变量 `p`。首先它是一个指针变量，所以要有一个“*”，即 `(*p)`；其次前面的 `int` 表示这个指针变量可以指向返回值类型为 `int` 型的函数；后面括号中的两个 `int` 表示这个指针变量可以指向有两个参数且都是 `int` 型的函数。所以合起来这个语句的意思就是：定义了一个指针变量 `p`，该指针变量可以指向返回值类型为 `int` 型，且有两个整型参数的函数。`p` 的类型为 `int(*) (int, int)`。

我们看到，函数指针的定义就是将“函数声明”中的“函数名”改成“(*指针变量名)”。但是这里需要注意的是：“(*指针变量名)”**两端的括号不能省略**，括号改变了运算符的优先级。如果省略了括号，就不是定义函数指针而是一个函数声明了，即声明了一个返回值类型为指针型的函数。

最后需要注意的是，指向函数的指针变量没有 `++` 和 `--` 运算。

```
# include <stdio.h>
int Max(int, int); //函数声明
int main(void)
{
    int(*p)(int, int); //定义一个函数指针
    int a, b, c;
    p = Max; //把函数Max赋给指针变量p，使p指向Max函数
    printf("please enter a and b:");
    scanf("%d%d", &a, &b);
    c = (*p)(a, b); //通过函数指针调用Max函数
    printf("a = %d\nb = %d\nmax = %d\n", a, b, c);
    return 0;
}
int Max(int x, int y) //定义Max函数
{
    int z;
    if (x > y)
    {
        z = x;
    }
    else
    {
        z = y;
    }
    return z;
}
```

2. 指针函数

首先它是一个函数，只不过这个函数的返回值是一个地址值。函数返回值必须用同类型的指针变量来接受，也就是说，指针函数一定有“函数返回值”，而且，在主调函数中，函数返回值必须赋给同类型的指针变量。

类型名 *函数名(函数参数列表);

其中，后缀运算符括号“()”表示这是一个函数，其前缀运算符星号“*”表示此函数为指针型函数，其函数值为指针，即它带回来的值的类型为指针，当调用这个函数后，将得到一个“指向返回值为...的指针（地址）”，“类型名”表示函数返回的指针指向的类型”。

“(函数参数列表)”中的括号为函数调用运算符，在调用语句中，即使函数不带参数，其参数表的一对括号也不能省略。其示例如下：

```
int *pfun(int, int);
```

由于“*”的优先级低于“()”的优先级，因而pfun首先和后面的“()”结合，也就意味着，pfun是一个函数。即：

```
int *(pfun(int, int));
```

接着再和前面的“*”结合，说明这个函数的返回值是一个指针。由于前面还有一个int，也就是说，pfun是一个返回值为整型指针的函数。

```
#include <stdio.h>
float *find(float(*pionter)[4],int n);//函数声明
int main(void)
{
    static float score[][4]={60,70,80,90},{56,89,34,45},{34,23,56,45}};
    float *p;
    int i,m;
    printf("Enter the number to be found:");
    scanf("%d",&m);
    printf("the score of NO.%d are:\n",m);
    p=find(score,m-1);
    for(i=0;i<4;i++)
        printf("%5.2f\t",*(p+i));

    return 0;
}

float *find(float(*pionter)[4],int n)/*定义指针函数*/
{
    float *pt;
    pt=*(pionter+n);
    return(pt);
}
```

共有三个学生的成绩，函数find()被定义为指针函数，其形参pointer是指针指向包含4个元素的一维数组的指针变量。pointer+n指向score的第n+1行。*(pointer+1)指向第一行的第0个元素。pt是一个指针变量，它指向浮点型变量。main()函数中调用find()函数，将score数组的首地址传给pointer。

数组名和指针的区别与联系是什么？

1. 数据保存方面

指针保存的是地址（保存目标数据地址，自身地址由编译器分配），内存访问偏移量为4个字节，无论其中保存的是何种数据均已地址类型进行解析。

数组保存的数据。数组名表示的是第一个元素的地址，内存偏移量是保存数据类型的内存偏移量；只有对数组名取地址（&数组名）时数组名才表示整个数组，内存偏移量是整个数组的大小（sizeof(数组名)）。

2. 数据访问方面

指针对数据的访问方式是间接访问，需要用到解引用符号（*数组名）。

数组对数据的访问则是直接访问，可通过下标访问或数组名+元素偏移量的方式

3. 使用环境

指针多用于动态数据结构（如链表，等等）和动态内存开辟。

数组多用于存储固定个数且类型统一的数据结构（如线性表等等）和隐式分配。

指针进行强制类型转换后与地址进行加法运算，结果是什么？

假设在32位机器上，在对齐为4的情况下,sizeof(long)的结果为4字节，sizeof(char*)的结果为4字节，sizeof(short int)的结果与 sizeof(short)的结果都为2字节，sizeof(char)的结果为1字节，sizeof(int)的结果为4字节，由于32位机器上是4字节对齐，以如下结构体为例：

```
struct BBB
{
    long num;

    char *name;

    short int data;

    char ha;

    short ba[5];

}*p;
```

当p=0x100000;则p+0x200=? (ulong)p+0x200=? (char*)p+0x200=?

其实,在32位机器下,`sizeof(struct BBB)=sizeof(*p)=4+4+2+2+1+3/*补齐*/+2*5+2/*补齐*/=24`字节,而 `p=0x100000`,那么`p+0x200=0x1000000+0x200*24` 指针加法,加出来的是指针所指类型的字节长度的整倍数,就是p偏移`sizeof(p)*0x200`。

`(ulong)p+0x200=0x10000010+0x200` 经过ulong后,已经不再是指针加法,而变成一个数值加法了。

`(char*)p+0x200=0x1000000+0x200*sizeof(char)` 结果类型是char*。

常量指针,指向常量的指针,指向常量的常量指针有什么区别?

1. 常量指针

```
int * const p
```

先看const再看*,是p是一个常量类型的指针,不能修改这个指针的指向,但是这个指针所指向的地址上存储的值可以修改。

2. 指向常量的指针

```
const int *p
```

先看*再看const,定义一个指针指向一个常量,不能通过指针来修改这个指针指向的值

3. 指向常量的常量指针

```
const int *const p
```

对于“指向常量的常量指针”,就必须同时满足上述1和2中的内容,既不可以修改指针的值,也不可以修改指针指向的值。

指针和引用的异同是什么?如何相互转换?

相同

1. 都是地址的概念,指针指向某一内存、它的内容是所指内存的地址;引用则是某块内存的别名。
2. 从内存分配上看:两者都占内存,程序为指针会分配内存,一般是4个字节;而引用的本质是指针常量,指向对象不能变,但指向对象的值可以变。两者都是地址概念,所以本身都会占用内存。

区别

1. 指针是实体,而引用是别名。
2. 指针和引用的自增(++)运算符意义不同,指针是对内存地址自增,而引用是对值的自增。
3. 引用使用时无需解引用(*),指针需要解引用;(关于解引用大家可以看看这篇博客,传送门)。

- 引用只能在定义时被初始化一次，之后不可变；指针可变。
- 引用不能为空，指针可以为空。
- “sizeof 引用”得到的是所指向的变量(对象)的大小，而“sizeof 指针”得到的是指针本身的大小，在32位系统指针变量一般占用4字节内存。

```
#include "stdio.h"

int main(){
    int x = 5;
    int *p = &x;
    int &q = x;

    printf("%d %d\n", *p, sizeof(p));
    printf("%d %d\n", q, sizeof(q));
}
```

```
//结果
5 8
5 4
```

由结果可知，引用使用时无需解引用(*)，指针需要解引用；我用的是64位操作系统，“sizeof 指针”得到的是指针本身的大小，及8个字节。而“sizeof 引用”得到的是的对象本身的大小及int的大小，4个字节。

转换

- 指针转引用**：把指针用*就可以转换成对象，可以用在引用参数当中。
- 引用转指针**：把引用类型的对象用&取地址就获得指针了。

```
int a = 5;
int *p = &a;
void fun(int &x){} //此时调用fun可使用 : fun (*p) ;
//p是指针，加个*号后可以转换成该指针指向的对象，此时fun的形参是一个引用值，
//p指针指向的对象会转换成引用x。
```

野指针是什么？

- 野指针是指向不可用内存的指针，当指针被创建时，指针不可能自动指向NULL，这时，默认值是随机的，此时的指针成为野指针。
- 当指针被free或delete释放掉时，如果没有把指针设置为NULL，则会产生野指针，因为释放掉的仅仅是指针指向的内存，并没有把指针本身释放掉。
- 第三个造成野指针的原因是指针操作超越了变量的作用范围。

如何避免野指针？

1. 对指针进行初始化。

```
//将指针初始化为NULL。
char * p = NULL;
//用malloc分配内存
char * p = (char *)malloc(sizeof(char));
//用已有合法的可访问的内存地址对指针初始化
char num[ 30] = {0};
char *p = num;
```

2. 指针用完后释放内存，将指针赋NULL。

```
delete(p);
p = NULL;
```

注：malloc函数分配完内存后需注意：

- a. 检查是否分配成功（若分配成功，返回内存的首地址；分配不成功，返回NULL。可以通过if语句来判断）
- b. 清空内存中的数据（malloc分配的空间里可能存在垃圾值，用memset或bzero 函数清空内存）

```
//s是 需要置零的空间的起始地址； n是 要置零的数据字节个数。
void bzero (void *s, int n) ;
// 如果要清空空间的首地址为p, value为值，size为字节数。
void memset(void *start, int value, int size);
```

C++中的智能指针是什么？

智能指针是一个类，用来存储指针（指向动态分配对象的指针）。

C++程序设计中使用堆内存是非常频繁的操作，堆内存的申请和释放都由程序员自己管理。程序员自己管理堆内存可以提高了程序的效率，但是整体来说堆内存的管理是麻烦的，C++11中引入了智能指针的概念，方便管理堆内存。使用普通指针，容易造成堆内存泄露（忘记释放），二次释放，程序发生异常时内存泄露等问题等，使用智能指针能更好的管理堆内存。

智能指针的内存泄漏如何解决？

为了解决循环引用导致的内存泄漏，引入了弱指针weak_ptr，weak_ptr的构造函数不会修改引用计数的值，从而不会对对象的内存进行管理，其类似一个普通指针，但是不会指向引用计数的共享内存，但是可以检测到所管理的对象是否已经被释放，从而避免非法访问。

预处理

预处理器标识#error的目的是什么？

`#error`预处理指令的作用是，编译程序时，只要遇到`#error`就会生成一个**编译错误提示消息，并停止编译**。其语法格式为：`#error error-message`。

下面举个例子：程序中往往有很多的预处理指令

```
#ifdef XXX
...
#else
#endif
```

当程序比较大时，往往有些宏定义是在外部指定的（如`makefile`），或是在系统头文件中指定的，当你不太确定当前是否定义了`XXX`时，就可以改成如下这样进行编译：

```
#ifdef XXX
...
#error "XXX has been defined"
#else
#endif
```

这样,如果编译时出现错误,输出了`XXX has been defined`,表明宏`XXX`已经被定义了。

定义常量谁更好？`#define`还是 `const`？

尺有所短，寸有所长，`define`与 `const`都能定义常量，效果虽然一样，但是各有侧重。

`define`既可以替代常数值，又可以替代表达式，甚至是代码段，但是容易出错，而 `const`的引入可以增强程序的可读性，它使程序的维护与调试变得更加方便。具体而言，它们的差异主要表现在以下3个方面。

1. `define`只是用来进行**单纯的文本替换**，`define`常量的**生命周期止于编译期**，**不分配内存空间**，它存在于程序的**代码段**，在实际程序中，它只是一个常数；而 `const`常量存在于程序的**数据段**，并在**堆栈中分配了空间**，`const`常量在程序中确实存在，并且可以被调用、传递
2. `const`常量有数据类型，而 `define`常量没有数据类型。编译器可以对 `const`常量进行类型安全检查，如类型、语句结构等，而 `define`不行。
3. 很多IDE支持调试 `const`定义的常量，而不支持 `define`定义的常量由于 `const`修饰的变量可以排除程序之间的不安全性因素，保护程序中的常量不被修改，而且对数据类型也会进行相应的检查，极大地提高了程序的**健壮性**，所以一般**更加倾向于用`const`来定义常量类型**。

`typedef`和 `define`有什么区别？

`typedef`与 `define`都是**替一个对象取一个别名**，以此来增强程序的可读性，但是它们在使用和作用上也存在着以下4个方面的不同。

1. 原理不同

`#define`是C语言中定义的语法，它是预处理指令，在预处理时进行简单而机械的字符串替换，**不做正确性检查**，不管含义是否正确照样代入，只有在编译已被展开的源程序时，才会发现可能的错误并报错。

例如，`# define PI 3.1415926`，当程序执行`area=Pr*r`语句时，PI会被替换为3.1415926。于是该语句被替换为`area=3.1415926*r*r`。如果把`# define`语句中的数字9写成了g，预处理也照样代入，而不去检查其是否合理、合法。

`typedef`是关键字，它在编译时处理，所以 `typedef`具有类型检查的功能。它在自己的作用域内给一个已经存在的类型一个别名，但是不能在一个函数定义里面使用标识符 `typedef`。例如，`typedef int INTEGER`，这以后就可用 `INTEGER`来代替`int`作整型变量的类型说明了，例如：`INTEGER a,b;`

用 `typedef`定义数组、指针、结构等类型将带来很大的方便，不仅使程序书写简单而且使意义更为明确，因而增强了可读性。例如：`typedef int a[10];`

表示a是整型数组类型，数组长度为10。然后就可用a说明变量，例如:语句`a s1,s2`；完全等效于语句 `int s1[10],s2[10]`。同理，`typedef void (*p) (void)`表示p是一种指向void型的指针类型。

2. 功能不同

`typedef`用来定义类型的别名，这些类型不仅包含内部类型（`int`、`char`等），还包括自定义类型（如 `struct`），可以起到使类型易于记忆的功能。

例如：`typedef int (*PF)(const char *, const char*)`

定义一个指向函数的指针的数据类型PF，其中函数返回值为`int`，参数为 `const char*`。`typedef`还有另外一个重要的用途，那就是定义机器无关的类型。例如，可以定义一个叫REAL的浮点类型，在目标机器上它可以获得最高的精度：`typedef long double REAL`，在不支持 `long double`的机器上，该 `typedef`看起来会是下面这样：`typedef double real`，在 `double`都不支持的机器上，该 `typedef`看起来会是这样：`typedef float REAL`。

`#define`不只是可以为类型取别名，还可以定义常量、变量、编译开关等。

3. 作用域不同

`#define`没有作用域的限制，只要是之前预定义过的宏，在以后的程序中都可以使用，而 `typedef`有自己的作用域。

程序示例如下：

```
void fun()
{
    #define A int
}
void gun()
{
    //这里也可以使用A，因为宏替换没有作用域，但如果上面用的是 typedef，那这里就不能用
    //A，不过，一般不在函数内使用 typedef
}
```

4. 对指针的操作不同

两者修饰指针类型时，作用不同。

```
#define INTPTR1 int*
typedef int* INTPTR2;
INTPTR1 p1, p2;
INTPTR2 p3, p4;
```

INTPTR1 p1, p2和INTPTR2 p3, p4的效果截然不同。INTPTR1 p1, p2进行字符串替换后变成`int*p1, p2`，要表达的意义是声明一个指针变量p1和一个整型变量p2。而INTPTR2 p3, p4，由于INTPTR2是具有含义的，告诉我们是—个指向整型数据的指针，那么p3和p4都为指针变量，这句相当于`int*p1, *p2`。从这里可以看出，进行宏替换是不含任何意义的替换，仅仅为字符串替换；而用typedef为一种数据类型起的别名是带有一定含义的。

程序示例如下

```
#define INTPTR1 int*
typedef int* INTPTR2
int a=1;
int b=2;
int c=3;
const INTPTR1 p1=&a;
const INTPTR2 p2=&b;
INTPTR2 const p3=&c;
```

上述代码中，const INTPTR1 p1表示p1是一个常量指针，即不可以通过p1去修改p1指向的内容，但是p1可以指向其他内容。而对于const INTPTR2 p2，由于INTPTR2表示的是个指针类型，因此用const去限定，表示封锁了这个指针类型，因此p2是一个指针常量，不可使p2再指向其他内容，但可以通过p2修改其当前指向的内容。INTPTR2 const p3同样声明的是一个指针常量。

如何使用define声明个常数，用以表明1年中有多少秒（忽略闰年问题）

```
#define SECOND_PER_YEAR (60*60*24*365)UL
```

#include <filename.h>和#include "filename.h"有什么区别？

对于#include <filename.h>，编译器先从标准库路径开始搜索filename.h，使得系统文件调用较快。而对于#include "filename.h"，编译器先从用户的工作路径开始搜索filename.h，然后去寻找系统路径，使得自定义文件较快。

头文件的作用有哪些？

头文件的作用主要表现为以下两个方面：

1. 通过头文件来调用库功能。出于对源代码保密的考虑，源代码不便（或不准）向用户公布，只要向用户提供头文件和二进制的库即可。用户只需要按照头文件中的接口声明来调用库功能，而不必关心接口是怎么实现的。编译器会从库中提取相应的代码。
2. 头文件能加强类型安全检查。当某个接口被实现或被使用时，其方式与头文件中的声明不一致，编译器就会指出错误，大大减轻程序员调试、改错的负担。

在头文件中定义静态变量是否可行，为什么？

不可行，如果在头文件中定义静态变量，会造成资源浪费的问题，同时也可能引起程序错误。因为如果在使用了该头文件的每个C语言文件中定义静态变量，按照编译的步骤，**在每个头文件中都会单独存在一个静态变量**，从而会引起**空间浪费**或者**程序错误**所以，不推荐在头文件中定义任何变量，当然也包括静态变量。

写一个"标准"宏MIN，这个宏输入两个参数并返回较小的一个？

```
#define MIN(A,B) ((A) <= (B) ? (A) : (B))
```

不使用流程控制语句，如何打印出1~1000的整数？

宏定义多层嵌套 (10 * 10 * 10)，printf多次输出。

```
#include <stdio. h>
#define B P,P,P,P,P,P,P,P,P,P
#define P L,L,L,L,L,L,L,L,L,L
#define L I,I,I,I,I,I,I,I,I,I,N
#define I printf ("%3d", i++)
#define N printf ("n")
int main()
{
    int i = 1;
    B;
    return 0;
}
```

简便写法,同样使用多层嵌套

```
#include<stdio. h>
#define A (x) x;x;x;x;x;x;x;x;x;x
int main ()
{
    int n=1;
    A(A(A(printf("%d", n++));
    return 0;
}
```

变量

全局变量和静态变量的区别是什么？

1. 全局变量的作用域为程序块，而局部变量的作用域为当前函数。

2. 内存存储方式不同，全局变量（静态全局变量，静态局部变量）分配在全局数据区（静态存储空间），后者分配在栈区。
3. 生命周期不同。全局变量随主程序创建而创建，随主程序销毁而销毁，局部变量在局部函数内部，甚至局部循环体等内部存在，退出就不存在了。
4. 使用方式不同。通过声明为全局变量，程序的各个部分都可以用到，而局部变量只能在局部使用。

全局变量可不可以定义在可被多个.C文件包含的头文件中？为什么？

可以，在不同的C文件中以static形式来声明同名全局变量。

可以在不同的C文件中声明同名的全局变量，前提是其中只能有一个C文件中对此变量赋初值，此时连接不会出错。

局部变量能否和全局变量重名？

能，局部会屏蔽全局。

局部变量可以与全局变量同名，在函数内引用这个变量时，会用到同名的局部变量，而不会用到全局变量。

对于有些编译器而言，在同一个函数内可以定义多个同名的局部变量，比如在两个循环体内都定义一个同名的局部变量，而那个局部变量的作用域就在那个循环体内。

函数

请写个函数在main函数执行前先运行

`__attribute__`可以设置函数属性（Function Attribute）、变量属性（Variable Attribute）和类型属性（Type Attribute）。

gnu对于函数属性主要设置的关键字如下：

```
alias:      设置函数别名。
aligned:    设置函数对齐方式。
always_inline/gnu_inline:
            函数是否是内联函数。
constructor/destructor:
            主函数执行之前、之后执行的函数。
format:
            指定变参函数的格式输入字符串所在函数位置以及对应格式输出的位置。
noreturn:
            指定这个函数没有返回值。
            请注意，这里的没有返回值，并不是返回值是void。而是像
            _exit/_exit/_abord那样
            执行完函数之后进程就结束的函数。
weak: 指定函数属性为弱属性，而不是全局属性，一旦全局函数名称和指定的函数名称
      命名有冲突，使用全局函数名称。
```

完整示例代码如下：


```

#include <stdio.h>

void before() __attribute__((constructor));
void after() __attribute__((destructor));

void before() {
    printf("this is function %s\n",__func__);
    return;
}

void after(){
    printf("this is function %s\n",__func__);
    return;
}

int main(){
    printf("this is function %s\n",__func__);
    return 0;
}

// 输出结果
// this is function before
// this is function main
// this is function after

```

为什么析构函数必须是虚函数？

将可能会被继承的父类的析构函数设置为虚函数，可以保证当我们new一个子类，然后使用基类指针指向该类对象，释放基类指针时可以释放掉子类的空间，防止内存泄漏。

为什么C++默认的析构函数不是虚函数？

C++默认的析构函数不是虚函数是因为虚函数需要额外的虚函数表和虚表指针，占用额外的内存。而对于不会被继承的类来说，其析构函数如果是虚函数，就会浪费内存。因此C++默认的析构函数不是虚函数，而是只有当需要当作父类时，设置为虚函数。

C++中析构函数的作用？

如果构造函数打开了一个文件，最后不需要使用时文件就要被关闭。析构函数允许类自动完成类似清理工作，不必调用其他成员函数。

析构函数也是特殊的类成员函数。简单来说，析构函数与构造函数的作用正好相反，它用来完成对象被删除前的一些清理工作，也就是专门的扫尾工作。

静态函数和虚函数的区别？

静态函数在编译的时候就已经确定运行时机，虚函数在运行的时候动态绑定。虚函数因为用了虚函数表机制，调用的时候会增加一次内存开销。

重载和覆盖有什么区别？

1. 覆盖是子类和父类之间的关系，垂直关系；重载同一个类之间方法之间的关系，是水平关系。
2. 覆盖只能由一个方法或者只能由一对方法产生关系；重载是多个方法之间的关系。
3. 覆盖是根据对象类型（对象对应存储空间类型）来决定的；而重载关系是根据调用的实参表和形参表来选择方法体的。

虚函数表具体是怎样实现运行时多态的？

原理：

虚函数表是一个类的虚函数的地址表，每个对象在创建时，都会有一个指针指向该类虚函数表，每一个类的虚函数表，按照函数声明的顺序，会将函数地址存在虚函数表中，当子类对象重写父类的虚函数的时候，父类的虚函数表中对应的位置会被子类的虚函数地址覆盖。

作用：

在用父类的指针调用子类对象成员函数时，虚函数表会指明要调用的具体函数是哪个。

C语言是怎么进行函数调用的？

大多数CPU上的程序实现使用栈来支持函数调用操作，栈被用来传递函数参数、存储返回信息、临时保存寄存器原有的值以备恢复以及用来存储局部变量。

函数调用操作所使用的栈部分叫做**栈帧结构**，每个函数调用都有属于自己的栈帧结构，栈帧结构由两个指针指定，帧指针（指向起始），栈指针（指向栈顶），函数对大多数数据的访问都是基于帧指针。下面是结构图：

栈指针和帧指针一般都有专门的寄存器，通常使用ebp寄存器作为帧指针，使用esp寄存器做栈指针。

帧指针指向栈帧结构的头，存放着上一个栈帧的头部地址，栈指针指向栈顶。

请你说一说select

1. select函数原型

```
int select(int maxfdp, fd_set *readfds, fd_set *writefds, fd_set *errorfds, struct
timeval *timeout);
```

2. 文件描述符的数量

单个进程能够监视的文件描述符的数量存在最大限制，通常是1024，当然可以更改数量；(在linux内核头文件中定义：#define __FD_SETSIZE 1024)

3. 就绪fd采用轮询的方式扫描

select返回的是int，可以理解为返回的是ready(准备好的)一个或者多个文件描述符，应用程序需要遍历整个文件描述符数组才能发现哪些fd句柄发生了事件，由于select采用轮询的方式扫描文件描述符（不知道那个文件描述符读写数据，所以需要把所有的fd都遍历），文件描述符数量越多，性能越差

4. 内核 / 用户空间内存拷贝

select每次都会改变内核中的句柄数据结构集(fd集合)，因而每次调用select都需要从用户空间向内核空间复制所有的句柄数据结构(fd集合)，产生巨大的开销

5. select的触发方式

select的触发方式是水平触发，应用程序如果没有完成对一个已经就绪的文件描述符进行IO操作，那么之后每次调用select还是会将这些文件描述符通知进程。

6. 优点

- a. select的可移植性较好，可以跨平台；
- b. select可设置的监听时间timeout精度更好，可精确到微秒，而poll为毫秒。

7. 缺点：

- a. select支持的文件描述符数量上限为1024，不能根据用户需求进行更改；
- b. select每次调用时都要将文件描述符集合从用户态拷贝到内核态，开销较大；
- c. select返回的就绪文件描述符集合，需要用户循环遍历所监听的所有文件描述符是否在该集合中，当监听描述符数量很大时效率较低。

请你说说fork,wait,exec函数

父进程产生子进程使用fork拷贝出来一个父进程的副本，此时只拷贝了父进程的页表，两个进程都读同一块内存，当有进程写的时候使用写实拷贝机制分配内存，exec函数可以加载一个elf文件去替换父进程，从此父进程和子进程就可以运行不同的程序了。fork从父进程返回子进程的pid，从子进程返回0.调用了wait的父进程将会发生阻塞，直到有子进程状态改变,执行成功返回0，错误返回-1。exec执行成功则子进程从新的程序开始运行，无返回值，执行失败返回-1。

数组

以下代码表示什么意思？

```
*(a[1]+1)、*(&a[1][1])、(*(a+1))[1]
```

第一个：因为a[1]是第2行的地址，a[1]+1偏移一个单位（得到第2行第2列的地址），然后解引用取值，得到a[1][1]；

第二个：[]优先级高，a[1][1]取地址再取值。

第三个：a+1相当于&a[1]，所以*(a+1)=a[1]，因此*(a+1)[1]=a[1][1]

数组下标可以为负数吗？

可以，因为下标只是给出了一个与当前地址的偏移量而已，只要根据这个偏移量能定位得到目标地址即可。下面给出一个下标为负数的示例：

数组下标取负值的情况：

```
#include <stdio.h>
int main()
{
    int i;
    int a[5]={0,1,2,3,4};
    int *p=&a[4]
    for (i=-4; i<=0; i++)
        printf("%d %x\n", p[i], &p[i]);
    return 0.
}
//输出结果为
//0 b3ecf480
//1 b3ecf484
//2 b3ecf488
//3 b3ecf48c
//4 b3ecf490
```

从上例可以发现，在C语言中，数组的下标并非不可以为负数，当数组下标为负数时，编译可以通过，而且也可以得到正确的结果，只是它表示的意思却是从当前地址**向前寻址**。

位操作

如何求解整型数的二进制表示中1的个数？

程序代码如下：

```
#include <stdio.h>
int func(int x)
{
    int countx = 0;
    while(x)
    {
        countx++;
        x = x&(x-1);
    }
    return countx;
}
int main()
{
    printf("%d\n",func(9999));
    return 0;
}
```

程序输出的结果为8。

在上例中，函数func () 的功能是将x转化为二进制数，然后计算该二进制数中含有的1的个数。首先以9为例来分析，9的二进制表示为1001,8的二进制表示为1000，两者执行&操作之后结果为1000，此时1000再与0111（7的二进制位）执行&操作之后结果为0。

为了理解这个算法的核心，需要理解以下两个操作：

- 1) 当一个数被减1时，它最右边的那个值为1的bit将变为0，同时其右边的所有的bit都会变成1。
- 2) 每次执行 $x \& (x-1)$ 的作用是把x对应的二进制数中的最后一位1去掉。因此，循环执行这个操作直到x等于0的时候，循环的次数就是x对应的二进制数中1的个数。

如何求解二进制中0的个数

图示分析(以25为例)：

```
int CountZeroBit(int num)
{
    int count = 0;

    while (num > 0)
    {
        count++;
        num |= (num + 1);    //算法转换
    }
    return count;
}

int main()
{
    int value = 25;
    int ret = CountZeroBit(value);
    printf("%d的二进制位中0的个数为%d\n", value, ret);
    system("pause");
    return 0;
}
```

交换两个变量的值，不使用第三个变量。即 $a=3, b=5$, 交换之后 $a=5, b=3$;

有两种解法, 一种用算术算法, 一种用 $\wedge$ (异或)。

```
a = a + b;
b = a - b;
a = a - b;
```

```
a = a^b; // 只能对int, char...
b = a^b;
a = a^b;
or
a ^= b ^= a;
```

给定一个整型变量a，写两段代码，第一个设置a的bit 3，第二个清除a的bit 3。在以上两个操作中，要保持其它位不变。

```
#define BIT3 (0x1<<3)
static int a;
void set_bit3(void)
{
    a |= BIT3;
}
void clear_bit3(void)
{
    a &= ~BIT3;
}
```

容器和算法

map和set有什么区别？分别又是怎么实现的？

map和set都是C++的关联容器，其底层实现都是红黑树（RB-Tree）。

由于 map 和set所开放的各种操作接口，RB-tree 也都提供了，所以几乎所有的 map 和set的操作行为，都只是转调 RB-tree 的操作行为。

map和set的区别在于：

map中的元素是**key-value**（键值对）对：**关键字起到索引的作用**，值则表示与索引相关联的数据；Set与之相对就是关键字的简单集合，**set中每个元素只包含一个关键字**。

set的迭代器是const的，不允许修改元素的值；map允许修改value，但不允许修改key。

其原因是因为map和set是根据关键字排序来保证其有序性的，如果允许修改key的话，那么首先需要删除该键，然后调节平衡，再插入修改后的键值，调节平衡，如此一来，严重破坏了map和set的结构，导致iterator失效，不知道应该指向改变前的位置，还是指向改变后的位置。所以**STL中将set的迭代器设置成const，不允许修改迭代器的值；而map的迭代器则不允许修改key值，允许修改value值。**

map支持下标操作，set不支持下标操作。

map可以用key做下标，map的下标运算符[]将关键码作为下标去执行查找，如果关键码不存在，则插入一个具有该关键码和mapped_type类型默认值的元素至map中，因此**下标运算符[]在map应用中需要慎用**，const_map不能用，只希望确定某一个关键值是否存在而不希望插入元素时也不应该使用，mapped_type类型没有默认值也不应该使用。如果find能解决需要，尽可能用find。

STL的allocator有什么作用？

STL的分配器用于封装STL容器在内存管理上的底层细节。在C++中，其内存配置和释放如下：

new运算分两个阶段：(1)调用::operator new配置内存;(2)调用对象构造函数构造对象内容

delete运算分两个阶段：(1)调用对象析构函数；(2)调用::operator delete释放内存

为了精密分工，STL allocator将两个阶段操作区分开来：内存配置有`alloc::allocate()`负责，内存释放由`alloc::deallocate()`负责；对象构造由`::construct()`负责，对象析构由`::destroy()`负责。

同时为了提升内存管理的效率，减少申请小内存造成的内存碎片问题，SGI STL采用了两级配置器，当分配的空间大小超过128B时，会使用第一级空间配置器；当分配的空间大小小于128B时，将使用第二级空间配置器。第一级空间配置器直接使用`malloc()`、`realloc()`、`free()`函数进行内存空间的分配和释放，而第二级空间配置器采用了内存池技术，通过空闲链表来管理内存。

STL迭代器如何删除元素？

对于序列容器`vector`、`deque`来说，使用`erase(iterator)`后，后边的每个元素的迭代器都会失效，但是后边每个元素都会往前移动一个位置，但是`erase`会返回下一个有效的迭代器；

对于关联容器`map`、`set`来说，使用了`erase(iterator)`后，当前元素的迭代器失效，但是其结构是红黑树，删除当前元素的，不会影响到下一个元素的迭代器，所以在调用`erase`之前，记录下一个元素的迭代器即可。

对于`list`来说，它使用了不连续分配的内存，并且它的`erase`方法也会返回下一个有效的`iterator`，因此上面两种正确的方法都可以使用。

STL中MAP数据如何存放的？

红黑树。`unordered_map`底层结构是哈希表

STL中map与unordered_map有什么区别？

`map`在底层使用了红黑树来实现，`unordered_map`是C++11标准中新加入的容器，它的底层是使用hash表的形式来完成映射的功能，`map`是按照`operator<`比较判断元素是否相同，以及比较元素的大小，然后选择合适的位置插入到树中。所以，如果对`map`进行遍历（中序遍历）的话，输出的结果是有序的。顺序就是按照`operator<`定义的大小排序。

而`unordered_map`是计算元素的Hash值，根据Hash值判断元素是否相同。所以，对`unordered_map`进行遍历，结果是无序的。

使用`map`时，需要为key定义`operator<`。而`unordered_map`的使用需要定义`hash_value`函数并且重载`operator==`。对于内置类型，如`string`，这些都不用操心，可以使用默认的。对于自定义的类型做key，就需要自己重载`operator<`或者`hash_value()`了。

所以说，当不需要结果排好序时，最好用`unordered_map`，插入删除和查询的效率要高于`map`。

vector和list的区别是什么？

1. `vector`底层实现是数组；`list`是双向 链表。
2. `vector`支持随机访问，`list`不支持。
3. `vector`是顺序内存，`list`不是。
4. `vector`在中间节点进行插入删除会导致内存拷贝，`list`不会。
5. `vector`一次性分配好内存，不够时才进行2倍扩容；`list`每次插入新节点都会进行内存申请。
6. `vector`随机访问性能好，插入删除性能差；`list`随机访问性能差，插入删除性能好。

STL中迭代器有什么作用？有指针为何还要迭代器？

1、迭代器

Iterator (迭代器) 模式又称Cursor (游标) 模式，用于提供一种方法顺序访问一个聚合对象中各个元素, 而又不需暴露该对象的内部表示。或者说可能更容易理解：Iterator模式是运用于聚合对象的一种模式，通过运用该模式，使得我们可以在不知道对象内部表示的情况下，按照一定顺序（由iterator提供的方法）访问聚合对象中的各个元素。

由于Iterator模式的以上特性：与聚合对象耦合，在一定程度上限制了它的广泛运用，一般仅用于底层聚合支持类，如STL的list、vector、stack等容器类及ostream_iterator等扩展iterator。

2、迭代器和指针的区别

迭代器不是指针，是类模板，表现的像指针。他只是模拟了指针的一些功能，通过重载了指针的一些操作符，->、*、++、--等。迭代器封装了指针，是一个“可遍历STL（Standard Template Library）容器内全部或部分元素”的对象，本质是封装了原生指针，是指针概念的一种提升（lift），提供了比指针更高级的行为，相当于一种智能指针，他可以根据不同类型的数据结构来实现不同的++，--等操作。

迭代器返回的是对象引用而不是对象的值，所以cout只能输出迭代器使用*取值后的值而不能直接输出其自身。

3、迭代器产生原因

Iterator类的访问方式就是把不同集合类的访问逻辑抽象出来，使得不用暴露集合内部的结构而达到循环遍历集合的效果。

epoll的原理是什么？

调用顺序：

```
int epoll_create(int size);
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
int epoll_wait(int epfd, struct epoll_event *events,int maxevents, int timeout);
```

首先创建一个epoll对象，然后使用epoll_ctl对这个对象进行操作，把需要监控的描述添加进去，这些描述如将会以epoll_event结构体的形式组成一颗红黑树，接着阻塞在epoll_wait，进入大循环，当某个fd上有事件发生时，内核将会把其对应的结构体放入到一个链表中，返回有事件发生的链表。

STL里resize和reserve的区别是什么？

改变当前容器内含有元素的数量(size())，eg: vector v; v.resize(len);v的 size 变为 len,如果原来 v 的 size 小于 len，那么容器新增 (len-size) 个元素，元素的值为 默认为 0.当 v.push_back(3);之后，则是 3 是放在了 v 的末尾，即下标为 len，此时容器是 size为 len+1；

改变当前容器的最大容量 (capacity) ,它不会生成元素，只是确定这个容器允许放入多少对象，如果 reserve(len)的值大于当前的 capacity()，那么会重新分配一块能存 len 个对象的空间，然后把之前 v.size()个对象通过 copy construtor 复制过来，销毁之前的内存；

类和数据抽象

C++中类成员的访问权限？

C++通过 `public`、`protected`、`private` 三个关键字来控制成员变量和成员函数的访问权限，它们分别表示公有的、受保护的、私有的，被称为成员访问限定符。在类的内部（定义类的代码内部），无论成员被声明为 `public`、`protected` 还是 `private`，都是可以互相访问的，没有访问权限的限制。在类的外部（定义类的代码之外），只能通过对象访问成员，并且通过对象只能访问 `public` 属性的成员，不能访问 `private`、`protected` 属性的成员。

C++中struct和class的区别是什么？

在C++中，可以用`struct`和`class`定义类，都可以继承。区别在于：`structural`的默认继承权限和默认访问权限是`public`，而`class`的默认继承权限和默认访问权限是`private`。另外，`class`还可以定义模板类形参，比如`template`。

C++类内可以定义引用数据成员吗？

可以，必须通过成员函数初始化列表初始化。

面向对象与泛型编程是什么？

1. 面向对象编程简称OOP,是一种程序设计思想。OOP把对象作为程序的基本单元,一个对象包含了数据和操作数据的函数。
2. 面向过程的程序设计把计算机程序视为一系列的命令集合，即一组函数的顺序执行。为了简化程序设计，面向过程把函数继续切分为子函数，即把大块函数通过切割成小块函数来降低系统的复杂度。
3. 泛型编程: 让类型参数化,方便程序员编码。类型参数化: 使的程序(算法)可以从逻辑功能上抽象,把被处理对象(数据)的类型作为参数传递。

什么是右值引用，跟左值又有什么区别？

左值和右值的概念：

左值：能对表达式取地址、或具名对象/变量。一般指表达式结束后依然存在的持久对象。

右值：不能对表达式取地址，或匿名对象。一般指表达式结束就不再存在的临时对象。

右值引用和左值引用的区别：

1. 左值可以寻址，而右值不可以。
2. 左值可以被赋值，右值不可以被赋值，可以用来给左值赋值。
3. 左值可变,右值不可变（仅对基础类型适用，用户自定义类型右值引用可以通过成员函数改变）。

析构函数可以为 `virtual` 型，构造函数则不能，为什么？

构造函数不能声明为虚函数，析构函数可以声明为虚函数，而且有时是必须声明为虚函数。不建议在构造函数和析构函数里面调用虚函数。

构造函数不能声明为虚函数的原因是:

虚函数的主要意义在于被派生类继承从而产生多态。派生类的构造函数中，编译器会加入构造基类的代码，如果基类的构造函数用到参数，则派生类在其构造函数的初始化列表中必须为基类给出参数，就是这个原因。虚函数的意思就是开启动态绑定，程序会根据对象的动态类型来选择要调用的方法。然而在构造函数运行的时

候，这个对象的动态类型还不完整，没有办法确定它到底是什么类型，故构造函数不能动态绑定。（动态绑定是根据对象的动态类型而不是函数名，在调用构造函数之前，这个对象根本就不存在，它怎么动态绑定？）

C++中空类默认产生哪些类成员函数？

C++中空类默认会产生以下6个函数：默认构造函数、复制构造函数、析构函数、赋值运算符重载函数、取址运算符重载函数、const取址运算符重载函数等。

```
class Empty
{
public:
    Empty(); // 缺省构造函数
    Empty( const Empty& ); // 拷贝构造函数
    ~Empty(); // 析构函数
    Empty& operator=( const Empty& ); // 赋值运算符
    Empty* operator&(); // 取址运算符
    const Empty* operator&() const; // 取址运算符 const
};
```

面向对象

面向对象和面向过程有什么区别？

面向对象与面向过程有以下四个方面的不同：

1. 出发点不同

面向对象使用符合常规思维的方式来处理客观世界的问题，强调把解决问题领域的“动作”直接映射到对象之间的接口上。而面向过程则强调的是过程的抽象化与模块化，是以过程为中心构造或处理客观世界问题。

2. 层次逻辑关系不同

面向对象使用计算机逻辑来模拟客观世界中的物理存在，以对象的集合类作为处理问题的单位，尽可能地使计算机世界向客观世界靠拢，以使处理问题的方式更清晰直接，面向对象使用类的层次结构来体现类之间的继承与发展。面向过程处理问题的基本单位是能清晰准确地表达过程的模块，用模块的层次结构概括模块或模块间的关系与功能，把客观世界的问题抽象成计算机可以处理的过程。

3. 数据处理方式与控制程序方式不同

面向对象将数据与对应的代码封装成一个整体，原则上其他对象不能直接修改其数据，即对象的修改只能由自身的成员函数完成，控制程序方式上是通过“事件驱动”来激活和运行程序的。而面向过程是直接通过程序来处理数据，处理完毕后即可显示处理的结果，在控制方式上是按照设计调用或返回程序，不能自由导航，各模块之间存在着控制与被控制，调动与被调用的关系。

4. 分析设计与编码转换方式不同

面向对象贯穿于软件生命周期的分析、设计及编码中，是一种平滑的过程，从分析到设计再到编码是采用一致性的模型表示，实现的是一种无缝连接。而面向过程强调分析、设计及编码之间按规则进行转换贯穿于软件生命周期的分析、设计及编码中，实现的是一种有缝的连接。

面向对象的基本特征有哪些？

面向对象的编程方法有四个基本特性：

1. 抽象：就是忽略一个主题中与当前目标无关的方面，以便更充分地注意与当前目标有关的方面。抽象并不打算了解全部问题，而只是选择其中的一部分，暂时不用部分细节。抽象包括两个方面，一是过程抽象，二是数据抽象。

过程抽象是指任何一个明确定义功能的操作都可被使用者看作单个的实体看待，尽管这个操作实际上可能由一系列更低级的操作来完成。数据抽象定义了数据类型和施加于该类型对象上的操作，并限定了对象的值，只能通过使用这些操作修改和观察。

2. 继承：这是一种联结类的层次模型，并且允许和鼓励类的重用，它提供了一种明确表述共性的方法。对象的一个新类可以从现有的类中派生，这个过程称为类继承。新类继承了原始类的特性，新类称为原始类的派生类（子类），而原始类称为新类的基类（父类）。

派生类可以从它的基类那里继承方法和实例变量，并且类可以修改或增加新的方法使之更适合特殊的需要。这也体现了大自然中一般与特殊的关系。继承性很好地解决了软件的可重用性问题。

3. 封装：就是把过程和数据包围起来，对数据的访问只能通过已定义的接口。面向对象的计算始于这个基本概念，即现实世界可以被描绘成一系列完全自治、封装的对象，这些对象通过一个受保护的接口访问其他对象。一旦定义了一个对象的特性，则有必要决定这些特性的可见性，即哪些特性对外部世界是可见的，哪些特性用于表示内部状态。

在这个阶段定义对象的接口。通常，应禁止直接访问一个对象的实际表示，而应通过操作接口访问对象，这称为信息隐藏。封装保证了模块具有较好的独立性，使得程序维护修改较为容易。对应用程序的修改仅限于类的内部，因而可以将应用程序修改带来的影响减少到最低限度。

4. 多态：是指允许不同类的对象对同一消息做出响应。比如同样的复制-粘贴操作，在字处理程序和绘图程序中有不同的效果。多态性包括参数化多态性和包含多态性。多态性语言具有灵活、抽象、行为共享、代码共享的优势，很好地解决了应用程序函数同名问题。

什么是深拷贝？什么是浅拷贝？

深拷贝是彻底的拷贝，两对象中所有的成员都是独立的一份，而且，成员对象中的成员对象也是独立一份。

浅拷贝中的某些成员变量可能是共享的，深拷贝如果不够彻底，就是浅拷贝。

什么是友元？

有成员只能在类的成员函数内部访问，如果想在别处访问对象的私有成员，只能通过类提供的接口（成员函数）间接地进行。这固然能够带来数据隐藏的好处，利于将来程序的扩充，但也会增加程序书写的麻烦。

C++是从结构化的C语言发展而来的，需要照顾结构化设计程序员的习惯，所以在对私有成员可访问范围的问题上不可限制太死。

C++ 设计者认为，如果有的程序员真的非常怕麻烦，就是想在类的成员函数外部直接访问对象的私有成员，那还是做一点妥协以满足他们的愿望为好，这也算是眼前利益和长远利益的折中。因此，C++ 就有了**友元（friend）**的概念。打个比方，这相当于是说：朋友是值得信任的，所以可以对他们公开一些自己的隐私。

友元提供了一种 普通函数或者类成员函数 访问另一个类中的私有或保护成员 的机制。也就是说有两种形式的友元：

(1) 友元函数：普通函数对一个访问某个类中的私有或保护成员。

(2) 友元类：类A中的成员函数访问类B中的私有或保护成员。

基类的构造函数/析构函数是否能被派生类继承？

基类的构造函数析构函数不能被派生类继承。

基类的构造函数不能被派生类继承，派生类中需要声明自己的构造函数。设计派生类的构造函数时，不仅要考虑派生类所增加的数据成员初始化，也要考虑基类的数据成员的初始化。声明构造函数时，只需要对本类中新增成员进行初始化，对继承来的基类成员的初始化，需要调用基类构造函数完成。

基类的析构函数也不能被派生类继承，派生类需要自行声明析构函数。声明方法与一般（无继承关系时）类的析构函数相同，不需要显式地调用基类的析构函数，系统会自动隐式调用。需要注意的是，析构函数的调用次序与构造函数相反。

初始化列表和构造函数初始化的区别？

构造函数初始化列表以一个冒号开始，接着是以逗号分隔的数据成员列表，每个数据成员后面跟一个放在括号中的初始化式。例如：

```
Example::Example() : ival(0), dval(0.0) {} //ival 和dval是类的两个数据成员
```

上面的例子和下面不用初始化列表的构造函数看似没什么区别：

```
Example::Example()
{
    ival = 0;
    dval = 0.0;
}
```

的确，这两个构造函数的结果是一样的。但区别在于：上面的构造函数（使用初始化列表的构造函数）显示的初始化类的成员；而没使用初始化列表的构造函数是对类的成员赋值，并没有进行显示的初始化。

初始化和赋值对内置类型的成员没有什么大的区别，像上面的任一个构造函数都可以。但有的时候必须用带有初始化列表的构造函数：

1. 成员类型是没有默认构造函数的类。若没有提供显示初始化式，则编译器隐式使用成员类型的默认构造函数，若类没有默认构造函数，则编译器尝试使用默认构造函数将会失败。
2. `const`成员或引用类型的成员。因为`const`对象或引用类型只能初始化，不能对他们赋值。

C++中有那些情况只能用初始化列表，而不能用赋值？

构造函数初始化列表以一个冒号开始，接着是以逗号分隔的数据成员列表，每个数据成员后面都跟一个放在括号中的初始化式。例如，`Example:Example ival (0,dva (0.0) {}`，其中`ival`与`dva`是类的两个数据成员。

在C++语言中，赋值与初始化列表的原理不一样，赋值是删除原值，赋予新值，初始化列表开辟空间和初始化是同时完成的，直接给予一个值

所以，在C++中，赋值与初始化列表的使用情况也不一样，只能用初始化列表，而不能用赋值的情况一般有以下3种：

1. 当类中含有 `const`（常量）、`reference`（引用）成员变量时，只能初始化，不能对它们进行赋值。常量不能被赋值，只能被初始化，所以必须在初始化列表中完成，C++的引用也一定要初始化，所以必须在初始化列表中完成。
2. 派生类在构造函数中要对自身成员初始化，也要对继承过来的基类成员进行初始化当基类没有默认构造函数的时候，通过在派生类的构造函数初始化列表中调用基类的构造函数实现。
3. 如果成员类型是没有默认构造函数的类，也只能使用初始化列表。若没有提供显式初始化时，则编译器隐式使用成员类型的默认构造函数，此时编译器尝试使用默认构造函数将会失败

类的成员变量的初始化顺序是什么？

1. 成员变量在使用初始化列表初始化时，与构造函数中初始化成员列表的顺序无关，只与定义成员变量的顺序有关。因为成员变量的初始化次序是根据变量在内存中次序有关，而内存中的排列顺序早在编译期就根据变量的定义次序决定了。这点在EffectiveC++中有详细介绍。
2. 如果不使用初始化列表初始化，在构造函数内初始化时，此时与成员变量在构造函数中的位置有关。
3. 注意：类成员在定义时，是不能初始化的
4. 注意：类中`const`成员常量必须在构造函数初始化列表中初始化。
5. 注意：类中`static`成员变量，必须在类外初始化。
6. 静态变量进行初始化顺序是基类的静态变量先初始化，然后是它的派生类。直到所有的静态变量都被初始化。这里需要注意全局变量和静态变量的初始化是不分次序的。这也不难理解，其实静态变量和全局变量都被放在公共内存区。可以把静态变量理解为带有“作用域”的全局变量。在一切初始化工作结束后，`main`函数会被调用，如果某个类的构造函数被执行，那么首先基类的成员变量会被初始化。

当一个类为另一个类的成员变量时，如何对其进行初始化？

示例程序如下：

```
class ABC
{
public:
    ABC(int x, int y, int z);
private:
    int a;
    int b;
    int c;
};

class MyClass
{
public:
    MyClass():abc(1,2,3)
```

```

    {

    }

private:
ABC abc;
};

```

上例中，因为ABC有了显式的带参数的构造函数，那么它是无法依靠编译器生成无参构造函数的，所以必须使用初始化列表:abc(1,2,3)，才能构造ABC的对象。

C++能设计实现一个不能被继承的类吗？

在Java 中定义了关键字final，被final 修饰的类不能被继承。但在C++ 中没有final 这个关键字，要实现这个要求还是需要花费一些精力。

首先想到的是在C++ 中，子类的构造函数会自动调用父类的构造函数。同样，子类的析构函数也会自动调用父类的析构函数。要想一个类不能被继承，我们只要把它的构造函数和析构函数都定义为私有函数。那么当一个类试图从它那继承的时候，必然会由于试图调用构造函数、析构函数而导致编译错误。

可是这个类的构造函数和析构函数都是私有函数了，我们怎样才能得到该类的实例呢？这难不倒我们，我们可以通过定义静态来创建和释放类的实例。基于这个思路，我们可以写出如下的代码：

```

/// // Define a class which can't be derived from  /// class FinalClass1
{
public :
static FinalClass1* GetInstance()
{
    return new FinalClass1;
}
static void DeleteInstance( FinalClass1* pInstance)
{
    delete pInstance;
    pInstance = 0;
}
private :
FinalClass1() {}
~FinalClass1() {}
};

```

这个类是不能被继承，但在总觉得它和一般的类有些不一样，使用起来也有点不方便。比如，我们只能得到位于堆上的实例，而得不到位于栈上实例。能不能实现一个和一般类除了不能被继承之外其他用法都一样的类呢？办法总是有的，不过需要一些技巧。请看如下代码：

```

/// // Define a class which can't be derived from  /// template <typename T>
class MakeFinal
{
    friend T;

```

```

private :
    MakeFinal() {}
    ~MakeFinal() {}
};
class FinalClass2 :
    virtual public MakeFinal<FinalClass2>
{
public :
    FinalClass2() {}
    ~FinalClass2() {}
};

```

这个类使用起来和一般的类没有区别，可以在栈上、也可以在堆上创建实例。尽管类 `MakeFinal` `<FinalClass2>` 的构造函数和析构函数都是私有的，但由于类 `FinalClass2` 是它的友元函数，因此在 `FinalClass2` 中调用 `MakeFinal` `<FinalClass2>` 的构造函数和析构函数都不会造成编译错误。但当我们试图从 `FinalClass2` 继承一个类并创建它的实例时，却不能通过编译。

```

class Try : public FinalClass2
{
public :
    Try() {}
    ~Try() {}
};    Try temp;

```

由于类 `FinalClass2` 是从类 `MakeFinal` `<FinalClass2>` 虚继承过来的，在调用 `Try` 的构造函数的时候，会直接跳过 `FinalClass2` 而直接调用 `MakeFinal` `<FinalClass2>` 的构造函数。非常遗憾的是 `Try` 不是 `MakeFinal` `<FinalClass2>` 的友元，因此不能调用其私有的构造函数。

基于上面的分析，试图从 `FinalClass2` 继承的类，一旦实例化，都会导致编译错误，因此是 `FinalClass2` 不能被继承。这就满足了我们设计要求。

构造函数没有返回值，那么如何得知对象是否构造成功？

这里的“构造”不单指分配对象本身的内存，而是指在建立对象时做的初始化操作（如打开文件、连接数据库等）。

因为构造函数没有返回值，所以通知对象的构造失败的唯一方法就是在构造函数中抛出异常。构造函数中抛出异常将导致对象的析构函数不被执行，当对象发生部分构造时，已经构造完毕的子对象将会逆序地被析构。

Public继承、protected继承、private继承的区别？

public（公有）继承、protected（保护）继承和 private（私有）继承是常见的3种继承方式。

1. 公有继承

对于子类的对象而言，采用公有继承时，基类成员对子类对象的可见性与一般类成员对对象的可见性相同，公有成员可见，其他成员不可见。

对于子类而言，基类的公有成员和保护成员可见；基类的公有成员和保护成员作为派生类的成员时，它们都维持原有的可见性（基类 `public` 成员在子类中还是 `public`，基类 `protected` 成员在子类中还是 `protected`）；基类的私有成员不可见，基类的私有成员依然是私有的，子类不可访问。

2. 保护继承

保护继承的特点是：基类的所有公有成员和保护成员都成为派生类的保护成员，并且只能被它的派生类成员函数或友元访问。基类的私有成员仍然是私有的。由此可以看出，基类的所有成员对子类的对象都是不可见的。

3. 私有继承

私有继承的特点是，基类的公有成员和保护成员都作为派生类的私有成员，并且不能被这个派生类的子类所访问。

C++提供默认参数的函数吗？

C++可以给函数定义默认参数值。在函数调用时没有指定与形参相对应的实参时，就自动使用默认参数。

默认参数的语法与使用：

(1) 在函数声明或定义时，直接对参数赋值，这就是默认参数。

(2) 在函数调用时，省略部分或全部参数。这时可以用默认参数来代替。

通常调用函数时，要为函数的每个参数给定对应的实参。例如：

```
void delay(int loops=1000); //函数声明

void delay(int loops) //函数定义
{
    if(loops==0)
    {
        return;
    }
    for(int i=0;i<loops;i++)
        ;
}
```

在上例中，如果将 `delay()` 函数中的 `loops` 定义成默认值 1000，这样，以后无论何时调用 `delay()` 函数，都不用给 `loops` 赋值，程序都会自动将它当做值 1000 进行处理。例如，当执行 `delay(2500)` 调用时，`loops` 的参数值为显性化的，被设置为 2500；当执行 `delay()` 时，`loops` 将采用默认值 1000。

默认参数在函数声明中提供，当有声明又有定义时，定义中不允许默认参数。如果函数只有定义，则默认参数才可出现在函数定义中。例如：

```
oid point(int=3, int=4); //声明中给出默认值

void point(int x, int y) //定义中不允许再给出默认值
{
    cout<<x<<endl;
```



```
    cout<<y<<endl;
}
```

如果一组重载函数（可能带有默认参数）都允许相同实参个数的调用，将会引起调用的二义性。例如：

```
void func(int); //重载函数之一
void func(int, int=4); //重载函数之二，带有默认参数
void func(int=3, int=4); //重载函数三，带有默认参数
func(7); //错误：到底调用3个重载函数中的哪个？
func(20,30); //错误：到底调用后面两个重载函数的哪个？
```

虚函数

什么是虚函数？

指向基类的指针在操作它的多态类对象时，可以根据指向的不同类对象调用其相应的函数，这个函数就是虚函数。

虚函数的作用：在基类定义了虚函数后，可以在派生类中对虚函数进行重新定义，并且可以通过基类指针或引用，在程序的运行阶段动态地选择调用基类和不同派生类中的同名函数。（如果在派生类中没有对虚函数重新定义，则它继承其基类的虚函数。）

下面是一个虚函数的实例程序：

```
#include "stdafx.h"
#include<iostream>
using namespace std;

class Base
{
public:
    virtual void Print()//父类虚函数
    {
        printf("This is Class Base!\n");
    }
};

class Derived1 :public Base
{
public:
    void Print()//子类1虚函数
    {
        printf("This is Class Derived1!\n");
    }
};

class Derived2 :public Base
{
public:
```

```

void Print()//子类2虚函数
{
    printf("This is Class Derived2!\n");
}

};

int main()
{
    Base Cbase;
    Derived1 Cderived1;
    Derived2 Cderived2;
    Cbase.Print();
    Cderived1.Print();
    Cderived2.Print();

    cout << "-----" << endl;
    Base *p1 = &Cbase;
    Base *p2 = &Cderived1;
    Base *p3 = &Cderived2;
    p1->Print();
    p2->Print();
    p3->Print();
}

/*
输出结果：
This is Class Base!
This is Class Derived1!
This is Class Derived2!
-----
This is Class Base!
This is Class Derived1!
This is Class Derived2!
*/

```

需要注意的是，虚函数虽然非常好用，但是在使用虚函数时，并非所有的函数都需要定义成虚函数，因为实现虚函数是有代价的。在使用虚函数时，需要注意以下几个方面的内容：

- (1) 只需要在声明函数的类体中使用关键字**virtual**将函数声明为虚函数，而定义函数时不需要使用关键字**virtual**。
- (2) 当将基类中的某一成员函数声明为虚函数后，派生类中的同名函数自动成为虚函数。
- (3) 非类的成员函数不能定义为虚函数，全局函数以及类的成员函数中静态成员函数和构造函数也不能定义为虚函数，但可以将析构函数定义为虚函数。
- (4) 基类的析构函数应该定义为虚函数，否则会造成内存泄漏。基类析构函数未声明**virtual**，基类指针指向派生类时，**delete**指针不调用派生类析构函数。有 **virtual**，则先调用派生类析构再调用基类析构。

C++如何实现多态？

C++中通过虚函数实现多态。虚函数的本质就是通过基类指针访问派生类定义的函数。每个含有虚函数的类，其实例对象内部都有一个虚函数表指针。该虚函数表指针被初始化为本类的虚函数表的内存地址。所以，在程

序中，不管对象类型如何转换，该对象内部的虚函数表指针都是固定的，这样才能实现动态地对对象函数进行调用，这就是C++多态性的原理。

纯虚函数指的是什么？

纯虚函数是一种特殊的虚函数，格式一般如下

```
class <类名>
{
    virtual()函数返回值类型 虚函数名 (形参表) =0;
    ...
};
class <类名>
```

由于在很多情况下，基类中不能对虚函数给出有意义的实现，只能把函数的实现留给派生类。例如，动物作为一个基类可以派生出老虎、孔雀等子类，但是动物本身生成对象不合情理，此时就可以将动物类中的函数定义为纯虚函数，如果基类中有纯虚函数，那么在子类中必须实现这个纯虚函数，否则子类将无法被实例化，也无法实现多态。

含有纯虚函数的类称为抽象类，抽象类不能生成对象。纯虚函数永远不会被调用，它们主要用来统一管理子类对象。

什么函数不能声明为虚函数？

常见的不能声明为虚函数的有：普通函数（非成员函数）；静态成员函数；内联成员函数；构造函数；友元函数。

1. 为什么C++不支持普通函数为虚函数？普通函数（非成员函数）只能被overload，不能被override，声明为虚函数也没有什么意思，因此编译器会在编译时绑定函数。

2. 为什么C++不支持构造函数为虚函数？这个原因很简单，主要是从语义上考虑，所以不支持。因为构造函数本来就是为了明确初始化对象成员才产生的，然而virtual function主要是为了再不完全了解细节的情况下也能正确处理对象。另外，virtual函数是在不同类型的对象产生不同的动作，现在对象还没有产生，如何使用virtual函数来完成你想完成的动作。（这不就是典型的悖论）

3. 为什么C++不支持内联成员函数为虚函数？其实很简单，那内联函数就是为了在代码中直接展开，减少函数调用花费的代价，虚函数是为了在继承后对象能够准确的执行自己的动作，这是不可能统一的。（再说了，inline函数在编译时被展开，虚函数在运行时才能动态的绑定函数）

4. 为什么C++不支持静态成员函数为虚函数？这也很简单，静态成员函数对于每个类来说只有一份代码，所有的对象都共享这一份代码，他也没有要动态绑定的必要性。

5. 为什么C++不支持友元函数为虚函数？因为C++不支持友元函数的继承，对于没有继承特性的函数没有虚函数的说法。

C++中如何阻止一个类被实例化？

C++中可以通过使用抽象类，或者将构造函数声明为private阻止一个类被实例化。抽象类之所以不能被实例化，是因为抽象类不能代表一类具体的事物，它是对多种具有相似性的具体事物的共同特征的一种抽象。例

如，动物作为一个基类可以派生出老虎、孔雀等子类，但是动物本身生成对象不合情理。